

```
In [9]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [10]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from torchvision import models, datasets
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
```

```
In [11]: ## Step 1: Load and Preprocess Data
# Define transformations for images
transform = transforms.Compose([
    transforms.Resize((224, 224)), # Resize images for pre-trained model input
    transforms.ToTensor(),
    #transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]) # Sta
])
```

```
In [14]: !unzip -qq ./chip_data.zip -d data
```

```
In [15]: # Load dataset from a folder (structured as: dataset/class_name/images)
dataset_path = "./data/dataset/"
train_dataset = datasets.ImageFolder(root=f"{dataset_path}/train", transform=transform)
test_dataset = datasets.ImageFolder(root=f"{dataset_path}/test", transform=transform)
```

```
In [16]: # Display some input images
def show_sample_images(dataset, num_images=5):
    fig, axes = plt.subplots(1, num_images, figsize=(5, 5))
    for i in range(num_images):
        image, label = dataset[i]
        image = image.permute(1, 2, 0) # Convert tensor format (C, H, W) to (H, W, C)
        axes[i].imshow(image)
        axes[i].set_title(dataset.classes[label])
        axes[i].axis("off")
    plt.show()
```

```
In [17]: # Show sample images from the training dataset
show_sample_images(train_dataset)
```



```
In [18]: # Get the total number of samples in the training dataset
print(f"Total number of training samples: {len(train_dataset)}")

# Get the shape of the first image in the dataset
first_image, label = train_dataset[0]
print(f"Shape of the first image: {first_image.shape}")
```

Total number of training samples: 172
Shape of the first image: torch.Size([3, 224, 224])

```
In [19]: # Get the total number of samples in the testing dataset

print("Number of testing samples:", len(test_dataset))

# Get the shape of the first image in the dataset

first_image1, label = test_dataset[0]
print("Image shape:", first_image1.shape)
```

Number of testing samples: 121
Image shape: torch.Size([3, 224, 224])

```
In [21]: # Create DataLoader for batch processing
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

```
In [23]: ## Step 2: Load Pretrained Model and Modify for Transfer Learning
# Load a pre-trained VGG19 model
# write your code here
from torchvision.models import VGG19_Weights
model = models.vgg19(weights=VGG19_Weights.DEFAULT)
```

Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.pth

100%|██████████| 548M/548M [00:03<00:00, 182MB/s]

```
In [25]: # Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

```
In [26]: from torchsummary import summary
# Print model summary
summary(model, input_size=(3, 224, 224))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 224, 224]	1,792
ReLU-2	[-1, 64, 224, 224]	0
Conv2d-3	[-1, 64, 224, 224]	36,928
ReLU-4	[-1, 64, 224, 224]	0
MaxPool2d-5	[-1, 64, 112, 112]	0
Conv2d-6	[-1, 128, 112, 112]	73,856
ReLU-7	[-1, 128, 112, 112]	0
Conv2d-8	[-1, 128, 112, 112]	147,584
ReLU-9	[-1, 128, 112, 112]	0
MaxPool2d-10	[-1, 128, 56, 56]	0
Conv2d-11	[-1, 256, 56, 56]	295,168
ReLU-12	[-1, 256, 56, 56]	0
Conv2d-13	[-1, 256, 56, 56]	590,080
ReLU-14	[-1, 256, 56, 56]	0
Conv2d-15	[-1, 256, 56, 56]	590,080
ReLU-16	[-1, 256, 56, 56]	0
Conv2d-17	[-1, 256, 56, 56]	590,080
ReLU-18	[-1, 256, 56, 56]	0
MaxPool2d-19	[-1, 256, 28, 28]	0
Conv2d-20	[-1, 512, 28, 28]	1,180,160
ReLU-21	[-1, 512, 28, 28]	0
Conv2d-22	[-1, 512, 28, 28]	2,359,808
ReLU-23	[-1, 512, 28, 28]	0
Conv2d-24	[-1, 512, 28, 28]	2,359,808
ReLU-25	[-1, 512, 28, 28]	0
Conv2d-26	[-1, 512, 28, 28]	2,359,808
ReLU-27	[-1, 512, 28, 28]	0
MaxPool2d-28	[-1, 512, 14, 14]	0
Conv2d-29	[-1, 512, 14, 14]	2,359,808
ReLU-30	[-1, 512, 14, 14]	0
Conv2d-31	[-1, 512, 14, 14]	2,359,808
ReLU-32	[-1, 512, 14, 14]	0
Conv2d-33	[-1, 512, 14, 14]	2,359,808
ReLU-34	[-1, 512, 14, 14]	0
Conv2d-35	[-1, 512, 14, 14]	2,359,808
ReLU-36	[-1, 512, 14, 14]	0
MaxPool2d-37	[-1, 512, 7, 7]	0
AdaptiveAvgPool2d-38	[-1, 512, 7, 7]	0
Linear-39	[-1, 4096]	102,764,544
ReLU-40	[-1, 4096]	0
Dropout-41	[-1, 4096]	0
Linear-42	[-1, 4096]	16,781,312
ReLU-43	[-1, 4096]	0
Dropout-44	[-1, 4096]	0
Linear-45	[-1, 1000]	4,097,000

Total params: 143,667,240

Trainable params: 143,667,240

Non-trainable params: 0

Input size (MB): 0.57

Forward/backward pass size (MB): 238.69
Params size (MB): 548.05
Estimated Total Size (MB): 787.31

```
In [27]: # Modify the final fully connected layer to match the dataset classes  
# Write your code here  
model.classifier[-1]=nn.Linear(model.classifier[-1].in_features,1)
```

```
In [28]: # Move model to GPU if available  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
model = model.to(device)
```

```
In [29]: summary(model, input_size=(3, 224, 224))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 224, 224]	1,792
ReLU-2	[-1, 64, 224, 224]	0
Conv2d-3	[-1, 64, 224, 224]	36,928
ReLU-4	[-1, 64, 224, 224]	0
MaxPool2d-5	[-1, 64, 112, 112]	0
Conv2d-6	[-1, 128, 112, 112]	73,856
ReLU-7	[-1, 128, 112, 112]	0
Conv2d-8	[-1, 128, 112, 112]	147,584
ReLU-9	[-1, 128, 112, 112]	0
MaxPool2d-10	[-1, 128, 56, 56]	0
Conv2d-11	[-1, 256, 56, 56]	295,168
ReLU-12	[-1, 256, 56, 56]	0
Conv2d-13	[-1, 256, 56, 56]	590,080
ReLU-14	[-1, 256, 56, 56]	0
Conv2d-15	[-1, 256, 56, 56]	590,080
ReLU-16	[-1, 256, 56, 56]	0
Conv2d-17	[-1, 256, 56, 56]	590,080
ReLU-18	[-1, 256, 56, 56]	0
MaxPool2d-19	[-1, 256, 28, 28]	0
Conv2d-20	[-1, 512, 28, 28]	1,180,160
ReLU-21	[-1, 512, 28, 28]	0
Conv2d-22	[-1, 512, 28, 28]	2,359,808
ReLU-23	[-1, 512, 28, 28]	0
Conv2d-24	[-1, 512, 28, 28]	2,359,808
ReLU-25	[-1, 512, 28, 28]	0
Conv2d-26	[-1, 512, 28, 28]	2,359,808
ReLU-27	[-1, 512, 28, 28]	0
MaxPool2d-28	[-1, 512, 14, 14]	0
Conv2d-29	[-1, 512, 14, 14]	2,359,808
ReLU-30	[-1, 512, 14, 14]	0
Conv2d-31	[-1, 512, 14, 14]	2,359,808
ReLU-32	[-1, 512, 14, 14]	0
Conv2d-33	[-1, 512, 14, 14]	2,359,808
ReLU-34	[-1, 512, 14, 14]	0
Conv2d-35	[-1, 512, 14, 14]	2,359,808
ReLU-36	[-1, 512, 14, 14]	0
MaxPool2d-37	[-1, 512, 7, 7]	0
AdaptiveAvgPool2d-38	[-1, 512, 7, 7]	0
Linear-39	[-1, 4096]	102,764,544
ReLU-40	[-1, 4096]	0
Dropout-41	[-1, 4096]	0
Linear-42	[-1, 4096]	16,781,312
ReLU-43	[-1, 4096]	0
Dropout-44	[-1, 4096]	0
Linear-45	[-1, 1]	4,097

Total params: 139,574,337

Trainable params: 139,574,337

Non-trainable params: 0

Input size (MB): 0.57

Forward/backward pass size (MB): 238.68
Params size (MB): 532.43
Estimated Total Size (MB): 771.69

```
In [30]: # Freeze all layers except the final layer
        for param in model.features.parameters():
            param.requires_grad = False # Freeze feature extractor layers
```

```
In [31]: # Include the Loss function and optimizer
        criterion = nn.BCEWithLogitsLoss()
        optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
In [44]: ## Step 3: Train the Model
        def train_model(model, train_loader, test_loader, num_epochs=100):
            train_losses=[]
            val_losses=[]
            model.train()
            for epoch in range(num_epochs):
                running_loss=0.0
                for images, labels in train_loader:
                    images, labels = images.to(device), labels.to(device)
                    optimizer.zero_grad()
                    outputs = model(images)
                    loss = criterion(outputs, labels.unsqueeze(1).float())

                    loss.backward()
                    optimizer.step()
                    running_loss += loss.item()
                train_losses.append(running_loss/len(train_loader))

                # Compute validation loss
                model.eval()
                val_loss=0.0
                with torch.no_grad():
                    for images, labels in test_loader:
                        images, labels = images.to(device), labels.to(device)
                        outputs = model(images)
                        loss = criterion(outputs, labels.unsqueeze(1).float())
                        val_loss += loss.item()
                val_losses.append(val_loss/len(test_loader))
            model.train()
            print(f'Epoch [{epoch+1}/{num_epochs}], Train Loss: {train_losses[-1]:

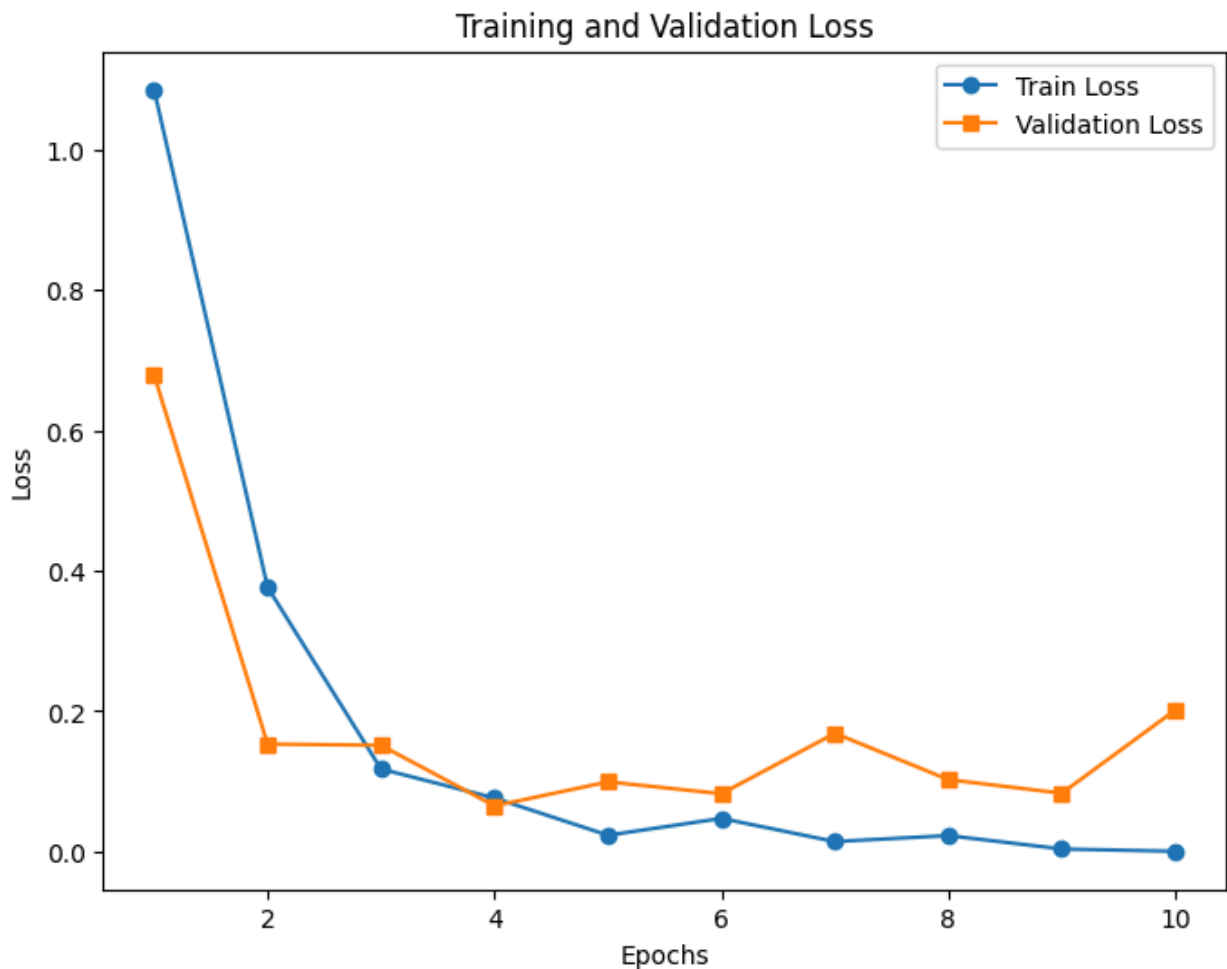
        # Plot training and validation loss
        print("Name: CH.V.S.DINESH KUMAR ")
        print("Register Number:212224040055")
        plt.figure(figsize=(8, 6))
        plt.plot(range(1, num_epochs + 1), train_losses, label='Train Loss', marker='x')
        plt.plot(range(1, num_epochs + 1), val_losses, label='Validation Loss', marker='o')
        plt.xlabel('Epochs')
        plt.ylabel('Loss')
        plt.title('Training and Validation Loss')
```

```
plt.legend()  
plt.show()
```

```
In [33]: # Move model to GPU if available  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
model = model.to(device)
```

```
In [34]: # Train the model  
# Write your code here  
  
train_model(model, train_loader, test_loader, num_epochs=10)
```

```
Epoch [1/10], Train Loss: 1.0861, Validation Loss: 0.6794  
Epoch [2/10], Train Loss: 0.3767, Validation Loss: 0.1529  
Epoch [3/10], Train Loss: 0.1174, Validation Loss: 0.1512  
Epoch [4/10], Train Loss: 0.0760, Validation Loss: 0.0648  
Epoch [5/10], Train Loss: 0.0229, Validation Loss: 0.0990  
Epoch [6/10], Train Loss: 0.0469, Validation Loss: 0.0822  
Epoch [7/10], Train Loss: 0.0139, Validation Loss: 0.1683  
Epoch [8/10], Train Loss: 0.0224, Validation Loss: 0.1022  
Epoch [9/10], Train Loss: 0.0033, Validation Loss: 0.0828  
Epoch [10/10], Train Loss: 0.0000, Validation Loss: 0.2015  
Name: CH.V.S.DINESH KUMAR  
Register Number:212224040055
```



In [45]: *## Step 4: Test the Model and Compute Confusion Matrix & Classification Report*

```
def test_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

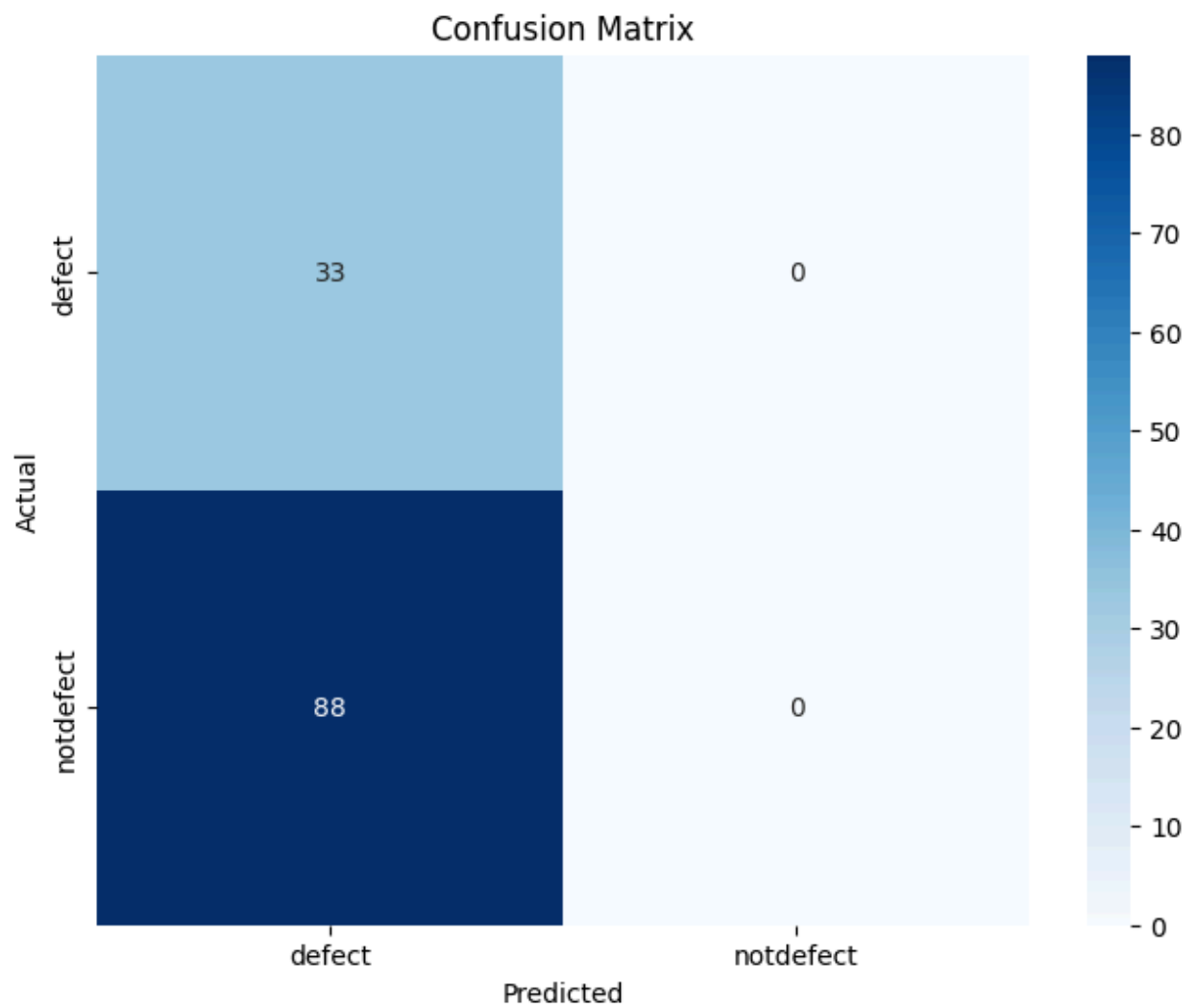
    accuracy = correct / total
    print(f'Test Accuracy: {accuracy:.4f}')

    # Compute confusion matrix
    cm = confusion_matrix(all_labels, all_preds)
    print("Name:CH.V.S.DINESH KUMAR")
    print("Register Number:212224040055")
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=train_data
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion Matrix')
    plt.show()

    # Print classification report
    print("Name:CH.V.S.DINESH KUMAR")
    print("Register Number:212224040055")
    print("Classification Report:")
    print(classification_report(all_labels, all_preds, target_names=train_data
```

In [46]: *# Evaluate the model*
write your code here
test_model(model,test_loader)

Test Accuracy: 0.2727
Name:CH.V.S.DINESH KUMAR
Register Number:212224040055



Name:CH.V.S.DINESH KUMAR
Register Number:212224040055
Classification Report:

	precision	recall	f1-score	support
defect	0.27	1.00	0.43	33
notdefect	0.00	0.00	0.00	88
accuracy			0.27	121
macro avg	0.14	0.50	0.21	121
weighted avg	0.07	0.27	0.12	121

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
5: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
5: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
5: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

```

In [41]: ## Step 5: Predict on a Single Image and Display It
def predict_image(model, image_index, dataset):
    model.eval()
    image, label = dataset[image_index]
    with torch.no_grad():
        image_tensor = image.unsqueeze(0).to(device)
        output = model(image_tensor)

        # Apply sigmoid to get probability, threshold at 0.5
        prob = torch.sigmoid(output)
        predicted = (prob > 0.5).int().item()

    class_names = class_names = dataset.classes
    # Display the image
    image_to_display = transforms.ToPILImage()(image)
    print("Name:CH.V.S.DINESH KUMAR")
    print("Register Number:212224040055")
    plt.figure(figsize=(4, 4))
    plt.imshow(image_to_display)
    plt.title(f'Actual: {class_names[label]}\nPredicted: {class_names[predicted]}')
    plt.axis("off")
    plt.show()

    print(f'Actual: {class_names[label]}, Predicted: {class_names[predicted]}')

```

```

In [42]: # Example Prediction
predict_image(model, image_index=55, dataset=test_dataset)

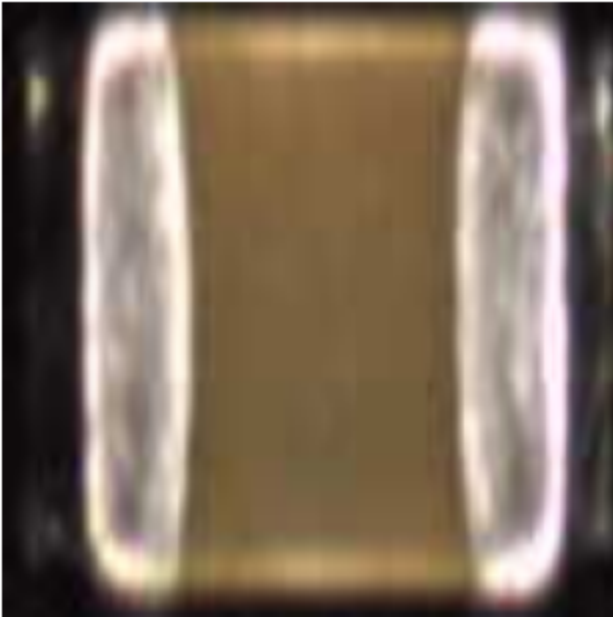
```

```

Name:CH.V.S.DINESH KUMAR
Register Number:212224040055

```

Actual: notdefect
Predicted: notdefect



Actual: notdefect, Predicted: notdefect

```
In [43]: #Example Prediction  
         predict_image(model, image_index=25, dataset=test_dataset)
```

Name:CH.V.S.DINESH KUMAR
Register Number:212224040055

Actual: defect
Predicted: defect



Actual: defect, Predicted: defect